

1 Oracle Dijkstra's

In some graph G , we are given a sorted list of nodes, sorted by their distances from some start vertex A . Design an *efficient* algorithm to find the shortest paths tree starting from A .

Hint: Your algorithm should be more efficient than Dijkstra's.

Just use this sorted list, no need to use PQ.

So there is no cost for PQ. Runtime $O(V+E)$ instead of
 $O(V \cdot \log V + E)$

2 Multiple MSTs

Recall a graph can have multiple MSTs if there are multiple spanning trees of minimum weight.

- (a) For each subpart below, select the correct option and justify your answer. If you select “never” or “always,” provide a short explanation. If you select “sometimes”, provide two graphs that fulfill the given properties — one with multiple MSTs and one without. Assume G is an undirected, connected graph.

1. If **none** the edge weights are **identical**, there will

- ☒ never be multiple MSTs in G .
☐ sometimes be multiple MSTs in G .
☐ always be multiple MSTs in G .

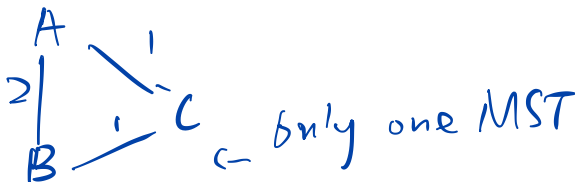
Justification:

leverage the cut property.

2. If **some** of the edge weights are **identical**, there will

- ☐ never be multiple MSTs in G .
☒ sometimes be multiple MSTs in G .
☐ always be multiple MSTs in G .

Justification:



3. If **all** of the edge weights are **identical**, there will

- ☐ never be multiple MSTs in G .
☒ sometimes be multiple MSTs in G .
☐ always be multiple MSTs in G .

Justification:

If it is a tree. No multiple.
 If there is cycle. Multiple.

- (b) Suppose we have a connected, undirected graph G with N vertices and N edges, where all the **edge weights are identical**. Find the maximum and minimum number of MSTs in G and explain your reasoning.

Minimum: 3

Maximum: N

Justification:

First build a tree that has N v and $N-1$ edges
and one more edge will cause a 3 edge cycle.
a 3 edge cycle can have three MST.

using N vertices build one circle. so it is a N edge
cycle. have N MST

- (c) It is possible that Prim's and Kruskal's find **different** MSTs on the same graph G (as an added exercise, construct a graph where this is the case!). Given any graph G with integer edge weights, modify G to **ensure** that Prim's and Kruskal's will always find the same MST. You may not modify Prim's or Kruskal's.

Hint: Look at subpart 1 of part a.

modify the edge to be unique.

but keep the weight.

We can modify them by adding $0.1, 0.2, 0.3 \dots$

And make sure all of $0 < 1$ and unique.

3 Graph Algorithm Design

For each of the following scenarios, write a brief description for an algorithm for finding the MST in an undirected, connected graph G .

- (a) If all edges have edge weight 1. Hint: Runtime is $O(V+E)$

BFS.

- (b) If all edges have edge weight 1 or 2. Hint: Use your algorithm from part (a)

Run BFS for 1. and Run Kruskal for all 2.

4 A Wordsearch

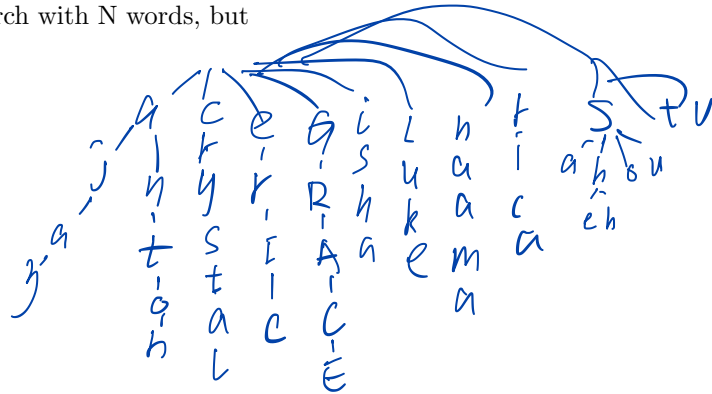
Given an N by N wordsearch and N words, devise an algorithm to solve the wordsearch in $O(N^3)$. For simplicity, assume no word is contained within another, i.e. if the word "bear" is given, "be" wouldn't also be given.

If you are unfamiliar with wordsearches or want to gain some wordsearch solving intuition, see below for an example wordsearch. Note that the below wordsearch doesn't follow the precise specification of an N by N wordsearch with N words, but your algorithm should work on this wordsearch regardless.

Example Wordsearch:

C	M	U	H	O	S	A	E	D
T	R	A	T	H	A	N	K	A
O	C	Y	E	S	R	T	U	T
N	I	R	S	A	I	O	L	S
Y	R	R	M	T	N	N	H	R
Y	E	A	E	V	A	R	U	E
A	A	A	I	M	E	L	C	R
N	H	D	J	Y	U	A	C	I
T	Y	S	A	A	R	S	U	C
A	R	S	I	G	Y	E	S	A

ajay	anton
crystal	eric
grace	isha
luke	naama
rica	sarina
sherry	shreyas
sohum	sumer
tony	vidya



Hint: Add the words to a **Trie**, and you may find the `longestPrefixOf` operation helpful. Recall that `longestPrefixOf` accepts a `String` key and returns the longest prefix of key that exists in the Trie, or `null` if no prefix exists.

We search all $N \times N$ letters.
 For each letter. Try to match the trie with path. cost
 is $O(N)$. for each letter.
 So total is $O(N^3)$